

# Examen Parcial de IA

(3 de mayo de 2018)

grupo 10

Duración: 55 min

1. (6 puntos) Tenemos que planificar la topología de interconexión de un conjunto de routers que están distribuidos por el Campus Nord, de manera que podamos canalizar todo su tráfico hacia el equipo de comunicaciones que lo reenvía a internet. Para cada router  $r_i$  sabemos su localización  $l_i$  en el campus (supondremos que tenemos un sistema de coordenadas en el que podemos situar cada router) y el ancho de banda  $bw_i$  del tráfico directo que debe distribuir (el que no le llega de otros routers). También conocemos las coordenadas del equipo de comunicaciones ( $l_{EC}$ ). Cada router puede estar conectado a otro router o al equipo de comunicaciones exterior directamente (la topología ha de ser un árbol). Disponemos de tres tipos de router (tipos A, B y C) capaces de distribuir hasta cierto ancho de banda máximo cada uno ( $bw_A < bw_B < bw_C$ ). Para distribuir el tráfico directo hay suficiente con un router de tipo A. El coste de cada router es proporcional al tipo (el tipo B cuesta el doble que el A y el C el doble que el B). Para conectar cada router  $r_i$  necesitamos instalar un cable  $c_i$  de cierta longitud (supondremos que es la distancia euclídea entre las coordenadas de los dos equipos conectados), el coste de este cable es proporcional a la longitud. El coste del equipo de comunicaciones exterior es fijo y no lo tendremos en cuenta.

El objetivo es decidir el tipo de los routers dependiendo del ancho de banda que deben soportar y la forma de interconectarlos, de manera que se que minimice el coste de la instalación.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (solución inicial, operadores, función heurística,...). Comenta muy brevemente la solución que se propone respecto a si es correcta, eficiente o mejor/peor respecto a otras alternativas posibles. Justifica tus respuestas.

- a) Usar Hill-climbing. Como solución inicial conectamos directamente todos los routers al equipo de comunicaciones exterior. Como operadores usamos conectar un router a otro y desconectar un router de otro. La función heurística es el sumatorio, para todos los routers, del coste del router más el coste del cable que lo conecta (a otro router o al equipo de comunicaciones).

HC es una técnica adecuada para este problema, ya que se requiere minimizar el coste económico de la instalación. Eso si, no asegura encontrar el óptimo.

La solución inicial es válida (es solución) si suponemos que se ponen todos los routers de tipo A. El coste de generación es  $O(R)$ , siendo  $R$  el número de routers. Es difícil estimar la calidad de esta solución inicial: aunque el coste de los routers es mínimo, la reducción del coste de las conexiones dependerá mucho de las posiciones de los routers.

Los operadores de conectar y desconectar son correctos, pero no permitirán obtener todas las soluciones. Falta un operador de cambio de tipo de un router, cuyo factor de ramificación sería  $O(R)$ . El operador conectar puede generar no-soluciones, ya que debería comprobar la capacidad del router al que se conecta para ver si podrá soportar la conexión a añadir, y comprobar que no se generan ciclos en las conexiones, su factor de ramificación es cuadrático respecto al número de routers ( $O(R^2)$ ). El operador desconectar siempre genera una no-solución y no hay restricciones que pueda comprobar, su factor de ramificación es  $O(R)$ . Para que se llegue a usar el operador desconectar hace falta una función heurística que se de cuenta de que esa desconexión nos acerca a una mejor solución. En este problema podríamos evitar esa dificultad usando un único operador de cambio de conexión en vez de los operadores desconectar y conectar.

La función heurística incluye exactamente los criterios que se quieren minimizar: el coste de los routers y el coste de los cables de conexión. Pero debería también penalizar las no-soluciones, aunque no de igual manera: los ciclos deberían tener una penalización muy grande, mientras que no debería penalizar una conexión a un router al que le falte capacidad

pero podamos incrementar el tipo, o una desconexión de un router si esos movimientos nos acercan a una solución. Al no penalizar no-soluciones y solo tener en cuenta los costes, la función heurística planteada hará que nunca se aplique el operador de cambio de tipo del router desde el tipo A inicial a los tipos más costosos, y solo aplicará el operador de desconectar router, ya que no solo es el único que reduce costes, sino que permite llegar al valor mínimo de la función heurística eliminando todas las conexiones, cosa que no nos interesa.

- b) Usar Hill-climbing. Como solución inicial conectamos cada router con el router más cercano y conectamos aleatoriamente uno de los routers al equipo exterior, asignamos a cada router el tipo A. Como operadores usamos cambiar la conexión de un router a otro router o equipo exterior por otra distinta si no superamos su capacidad y cambiar el tipo de un router. La función heurística quiere reducir al máximo la longitud de las conexiones y por ello es el sumatorio, para todos los routers, del coste del router más el coste del cable que lo conecta a otro router o al equipo de comunicaciones dividido por la longitud de esa conexión.

Esta solución inicial puede no ser válida, dependiendo de como estén colocados los routers, ya que podríamos tener algún router de tipo A desbordado por un exceso de conexiones, o tener un grupo de routers cercanos entre sí y apartados del resto que podrían provocar islas de routers conectados entre si y desconectados del resto, e incluso se podría producir algún ciclo de conexiones. El coste de generación es cuadrático respecto al número de routers ( $O(R^2)$ ) puesto que, para cada uno de los routers hemos de buscar cual de todos los otros routers es el más cercano. La calidad de la solución es media o baja (si es solución), ya que aunque esta solución inicial reduce el precio de las conexiones entre routers, hay margen de mejora para escoger mejor el router que se conecta al equipo de comunicaciones, y para escoger el tipo de router adecuado.

Los operadores permiten cubrir todo el espacio de soluciones. El operador de cambio de conexión debería comprobar la topología de conexiones para no generar ciclos, y la capacidad de un router para admitir una conexión más, su factor de ramificación es  $O(R^2)$ . El operador de cambio del tipo de un router debería comprobar que el tipo a asignar puede soportar las conexiones que recibe el router, su factor de ramificación es  $O(R^2)$ . Tal y como está el heurístico nunca se aplicará el operador de cambio a un tipo superior ya que se incrementa el coste, y por ello sería mejor que el cambio de tipo fuera automático dentro del operador de cambio de conexión.

La función heurística de nuevo incluye los criterios a minimizar, pero añade una división que no tiene sentido, por varias razones. La primera es que, si minimizamos el heurístico, al poner en el denominador la longitud de las conexiones el heurístico tendrá valores menores con longitudes mayores de conexión, que es lo contrario de lo que nos interesa. Es por ello que la longitud de la conexiones debería estar en el numerador, y de hecho ya lo está, forma parte del cálculo del coste de cada conexión. La segunda razón es que al formar parte del coste de las conexiones en el numerador, tenemos la longitud de las conexiones en numerador y denominador a la vez, por lo que se anulan y el heurístico intentará minimizar solo el coste de los routers. Esto haría que nunca se aplicara el operador de cambio de conexiones.

- c) Usar Algoritmos Genéticos. Supondremos que los routers y el equipo exterior están numerados consecutivamente, supondremos que para codificar ese número hacen falta  $n$  bits. La codificación es una tira  $r \cdot n$  de bits, donde  $r$  es el número de routers, cada grupo de  $n$  bits corresponden a un router siguiendo el orden de la numeración (es importante tener en cuenta que  $r + 1$  puede ser menor que  $2^n$ ). Como operadores usamos los operadores de cruce y mutación habituales. Para generar la población inicial usamos el método del apartado b) . La función heurística es la suma ponderada del coste de cada router (con peso 0,7) y el coste de su cable de conexión (con peso 0,3) para darle mayor relevancia al coste del router (que es lo más caro).

AG es una técnica adecuada para este problema ya que nos piden minimizar criterios y en este dominio hay una cierta dificultad para generar heurísticos que no tengan mesetas o que no empeoren momentaneamente para acercarnos a una solución (las mesetas y los mínimos locales son problemáticos para HC, pero no para AG). Eso si, al igual que en los apartados anteriores, no podemos asegurar el encontrar el óptimo pero no nos lo piden.

La representación es eficiente en espacio, y permite representar todas las topologías correctas de conexiones, pero también permite representar no-soluciones (por ejemplo, identificadores de router inválidos o ciclos de conexiones). Otro problema es que no representa el tipo de cada router.

Como se comentó ya en el apartado anterior, ese generador de soluciones iniciales puede generar no-soluciones o soluciones de calidad media. Para que sea útil para el algoritmo genético, lo hemos de llamar más de una vez para poder generar la población de soluciones iniciales que requiere la técnica. El coste de generación de esta población sería muy elevado ( $O(R^2 \cdot N)$ , siendo  $N$  el número de individuos de la población inicial) para la poca variabilidad que ofrece, ya que todos los individuos serían iguales excepto por la conexión random que se establece desde uno de los routers al equipo de comunicaciones.

Tanto el operador de cruce como el de mutación pueden crear no-soluciones, ya que pueden crear identificadores de router inválidos, sobrecargar routers o generar topologías que no son arbóreas (ciclos de conexiones, zonas inconexas). Y es un problema difícil de arreglar con los operadores típicos de cruce y mutación (podríamos limitar el operador de cruce para que no escoja un punto en medio de la codificación de un router, pero poco más). Y como la codificación no incluye los tipos de router, ninguno de los operadores puede generar un cambio de tipo de router.

Dado que tanto la población inicial como los operadores pueden fácilmente situarnos fuera del espacio de soluciones, la función heurística debería penalizar las no-soluciones y promover las soluciones mejores. Aparte de esto, en esta versión la función introduce una ponderación entre los costes que es innecesaria, ya que el coste de los routers y el coste de las conexiones están en las mismas unidades (euros) y tan importante es un euro de router como un euro de cable. En el enunciado se pide minimizar el coste global de la instalación, y la ponderación no aporta ninguna mejora en este sentido.

2. (4 puntos) La cadena de comida rápida a domicilio ComidApp ha decidido renovar su flota de motos de reparto por motos 100 % eléctricas pero después de unas pruebas ha visto que necesita hacer una mejor gestión de los trayectos de entrega para reducir la distancia recorrida y sacar el máximo partido de la limitada autonomía de las motos. Para ello nos piden un sistema que decida que moto y en que orden ha de servir los pedidos realizados por los clientes desde sus casas con una app. Esta app ya envía directamente el pedido (con su localización GPS) a la tienda más cercana al cliente.

Cada tienda tiene 4 motos eléctricas para entregar los pedidos asignados a su tienda. Cada moto puede llevar un máximo de 5 pedidos a la vez. Dado que no pueden tener las motos paradas durante horas cargando sus baterías al máximo, después de hacer pruebas han visto que lo mejor es hacer cargas rápidas de 10 minutos cada vez que la moto vuelve a la tienda, dando así una autonomía máxima de 8 km. Disponemos de una función `distancia(loc_1, loc_2)` que es capaz de calcular la longitud (en km) de la ruta más corta entre dos localizaciones GPS (tanto de clientes como de la tienda). Nos piden un sistema que, cada 30 minutos, asigne los pedidos recibidos por la tienda a las motos. Cada moto recibe una lista de pedidos a servir con el orden de entrega, y debe tener en cuenta que la longitud de la ruta (de la tienda al primer cliente, del primer cliente al segundo, del segundo al tercero... y del último a la tienda) no supere la autonomía máxima. Toda solución propuesta ha de entregar todos los pedidos.

- a) Queremos utilizar  $A^*$  para minimizar el número de kilómetros recorridos por las motos para asegurar que entregan todos los pedidos sin superar su autonomía máxima. El operador es asignar un pedido de la tienda a una moto (el orden de asignación determina el orden de entrega), el coste de ese operador es la distancia desde la localización donde se encuentra la moto a la localización de entrega de ese pedido (usando la función `distancia(loc_1, loc_2)` antes mencionada). La función heurística vale infinito si a alguna de las motos se le asigna más de 5 pedidos o si alguna de las motos no tiene energía suficiente para volver desde la localización actual a la tienda, y en caso contrario es la suma de las distancias de las localizaciones de los pedidos por entregar a la localización de la moto más cercana en cada caso.

Estamos planteando el problema como la construcción de una solución paso a paso a partir del estado inicial vacío minimizando algún criterio, lo que es adecuado para el algoritmo  $A^*$ . Pero aunque está bien planteado,  $A^*$  no es la técnica adecuada para este problema: el problema pide reducir la distancia recorrida por las motos para poder servir los pedidos con la autonomía limitada de dichas motos, mientras que el objetivo de un  $A^*$  es encontrar el óptimo (la distancia mínima), algo que no nos piden y que incrementa innecesariamente la complejidad de la resolución del problema.

Aunque no se dice de forma explícita, por el operador propuesto podemos deducir que el estado representa las asignaciones de pedidos a las motos con un orden dado.

El operador de asignar pedido (asumiendo estado inicial vacío) es correcto, va construyendo correctamente la ruta de las motos añadiéndoles pedidos de uno en uno, el factor de ramificación es  $O(M \cdot P)$ , siendo  $M = 4$  el número de motos y  $P$  el número de pedidos de la tienda. El operador no controla ninguna restricción antes de aplicarse, pero todo está controlado por las funciones  $g$  y  $h$ : en principio no se dice que el operador no pueda volver a asignar un pedido ya asignado, pero si lo hiciera añadiríamos el coste de ir a la localización del pedido en  $g$  y no se reduciría la distancia de pedidos por asignar en  $h$ , así que ese camino hacia la solución perdería delante de otras opciones mejores; si al asignar pedido a la moto excedemos los 5 pedidos o la moto se queda sin autonomía suficiente para volver a la tienda,  $h$  penaliza esta opción valiéndola infinito.

La función  $g$  es la acumulación de los costes del operador, por lo que acumula la distancia que recorre cada moto con los pedidos ya asignados. La función heurística  $h$  intenta estimar la distancia que falta por recorrer para servir los pedidos sin asignar, por lo que ambas funciones usan las mismas unidades y están alineadas. Como ya se ha comentado antes,

tiene sentido que  $h$  tome el valor infinito cuando el operador añade un pedido que viola alguna de las restricciones del problema. Y está bien planteado que  $h$  solo valga cero cuando todos los pedidos estén asignados (de esta forma fuerza al algoritmo hacia estados donde estén todos los pedidos asignados). El problema es como hace la estimación de la distancia que queda por recorrer a las motos. Existen casos en los que el heurístico sobreestima la distancia real, como por ejemplo: tenemos 4 pedidos con localizaciones muy próximas entre si (a 100 metros de distancia entre ellas), la moto más próxima esta a 900 metros\* de distancia de esas localizaciones (\* distancia media, con una variación de +20 o -20 metros). La distancia mínima ( $h^*$ ) para recorrer los pedidos es  $1 \cdot 900m + 3 \cdot 100m = 1200m$  mientras que el heurístico  $h$  nos daría  $4 \cdot 900m = 3600m$ . En este caso  $h > h^*$  y por lo tanto  $h$  no es admisible. En este problema es difícil encontrar una función  $h$  admisible y suficientemente informada para reducir significativamente el coste computacional de la búsqueda ya que estimar la distancia real del mínimo camino restante no es sencillo.

- b) Queremos utilizar satisfacción de restricciones para encontrar una asignación correcta de pedidos a motos. Asumiendo que cada moto puede llevar como máximo 5 pedidos, tenemos, para cada moto, cinco variables que representan el primer/segundo/tercer/cuarto/quinto pedido a entregar, el dominio para todas esas variables es la lista de identificadores de todos los pedidos recibidos por la tienda. Tenemos una restricción binaria  $\neq$  entre todo par de variables que impide que el mismo identificador de pedido se asigne a 2 o más variables a la vez. Tenemos una restricción n-aria entre las 5 variables de cada moto que impide que la longitud de la ruta necesaria para entregar los pedidos exceda la autonomía máxima de esa moto.

Satisfacción de restricciones es una técnica adecuada para este problema ya que nos piden una solución que reduzca lo suficiente la distancia recorrida para que las motos puedan servir todos los pedidos con su autonomía limitada, y eso es algo que podemos hacer con esta técnica: encontrar una asignación de pedidos a motos que sea solución cumpliendo las restricciones de autonomía y capacidad de las motos.

Tenemos  $4 \cdot 5$  variables que representan los pedidos asignados a las motos y el orden entre pedidos. El dominio de todas estas variables son los pedidos a asignar, lo cual no es del todo correcto, ya que si hemos de reducir todas las variables a un solo valor, significa que cada moto ha de repartir exactamente 5 pedidos, ni uno menos. Para evitar este problema y permitir que las motos puedan repartir menos de 5 pedidos, hay que añadir el valor *nulo* o *indefinido* a los dominios de todas las variables. Con esta pequeña modificación, y suponiendo que tenemos representadas correctamente todas las restricciones, una asignación de valores a todas las variables que cumpla las restricciones será siempre solución.

La limitación de que cada moto puede llevar 5 pedidos como máximo está garantizada por las variables escogidas. La restricción de que un mismo pedido no puede ser asignado más de una vez está resuelta con las restricciones binarias de desigualdad entre todo par de variables, y son fáciles de aplicar (se podría aplicar por ejemplo forward checking sobre los dominios de las variables futuras). La restricción de que los pedidos asignados a una moto no excedan la autonomía máxima de la misma está cubierta con las 4 restricciones n-arias entre las 5 variables de cada moto, y al ser cotas máximas son restricciones que se pueden comprobar desde el inicio del algoritmo y que podarán asignaciones que superen el máximo, por lo que sirven para guiar el algoritmo. Solo falta una restricción global que abarque todas las variables y que asegure que todo pedido está asignado a alguna moto. Esta restricción no la podemos mirar hasta las últimas asignaciones, por lo que puede generar muchos backtrackings.

Comenta cada una de las posibilidades indicando si resuelven o no el problema, qué errores te parece que tiene cada solución y cómo se podrían corregir, y qué ventajas e inconvenientes tienen cada una de ellas. Justifica la respuesta.